

1. Introduction

1.1 Background

Lithium-ion batteries lose capacity with cycling. In electric mobility and grid storage, estimating **how long a cell will last** without completing full life testing (often hundreds to thousands of cycles) reduces development time and cost. Severson et al. (2019) showed that **early-cycle** charge/discharge data can predict **cycle life** before visible capacity fade using machine learning on a large fast-charging dataset.

This project implements a reproducible pipeline on that public dataset: extract labels and per-cycle tables, engineer early-cycle features, train classical ML and one sequence model, and evaluate a simple monotonic state-of-health (SOH) constraint within an eight-week GNEM research plan.

1.2 Problem definition

Input: Per-cycle measurements from the first N cycles ($N = 20, 50$, or 100 in ablation studies), including discharge capacity, temperature, internal resistance, and related summary statistics.

Output: End-of-life (EOL) — the cycle index when discharge capacity first falls below **80%** of initial capacity (initial measured at the first cycle with `cycle_index` ≥ 1 , excluding formation cycle 0).

Derived target: Remaining useful life at cycle k : $RUL = EOL - k$.

1.3 Research questions

1. Can early-cycle summary data predict EOL before significant fade?
2. Which features from the first 20, 50, and 100 cycles are most informative?
3. Does an LSTM/GRU on early sequences outperform ML on hand-crafted features?
4. Does a monotonic SOH penalty improve physical plausibility of predicted degradation curves?

1.4 Scope

In scope: Public MIT-Stanford-Toyota data; Jupyter/Python pipeline; interpretable features; sklearn baselines; one LSTM/GRU; light monotonic regularizer; quantitative evaluation and report.

Out of scope: New experiments; full electrochemical or PINN models; Transformer/foundation models; BMS deployment.

1.5 Report organization

Section 2 reviews related work. Section 3 describes data and exploratory analysis. Sections 4–6 (Weeks 3–7) cover features, methods, and results. Section 7 discusses limitations; Section 8 concludes.

2. Related Work

2.1 Early cycle-life prediction

Severson et al. (2019) introduced the MIT-Stanford-Toyota fast-charging dataset and demonstrated **cycle-life prediction from early cycles** using hand-crafted features (including voltage–capacity curve differences) and a regularized linear model, achieving accurate ranking before substantial degradation. This work defines both our **primary dataset** and the core prediction task.

Fei et al. (2021) proposed a three-stage ML framework: **42 features** from the first 100 cycles, feature selection, and multiple regressors (SVM, elastic net, GPR), reporting strong early-lifetime accuracy. That framework informs our Week 3–4 feature engineering and baseline model choices.

2.2 Public battery data

dos Reis et al. (2021) surveyed **30+ public** lithium-ion datasets (NASA, CALCE, Oxford, etc.), cataloging test modes, variables (capacity, temperature, impedance), and ML use cases (SOH, SOC, RUL). We use this review to justify dataset selection and to note optional supplemental validation sets.

2.3 Deep learning and physics-informed approaches

Chen et al. (2022) applied a **Transformer** with a denoising autoencoder for RUL prediction—relevant as comparison context for our planned LSTM/GRU (Week 5). Wang et al. (2024) used a **physics-informed neural network** for SOH estimation across 387 cells, noting **80% SOH as first service life**—motivating our Week 6 monotonic SOH constraint without full PINN complexity.

Li et al. (2025) showed that models fit to capacity fade alone can be **non-unique** without degradation-mode analysis—a limitation we acknowledge for mechanism-agnostic ML. Hu et al. (2025) (BatteryGPT) represents state-of-the-art generative early prediction and is cited as **future work** outside our eight-week scope.

2.2 Summary

Our project sits between Severson-style **interpretable early prediction** and modern **sequence/physics-informed** methods: reproducible data pipeline, classical ML

baselines, one sequence model, and a simple physical consistency check on SOH trajectories.

Full reference list: see [09_references.md](#).

3. Data and Exploratory Analysis

3.1 Source

We use the **MIT-Stanford-Toyota** fast-charging dataset (Severson et al., 2019), downloaded from MATR.io. It contains commercial **LFP/graphite** pouch cells cycled under varied fast-charge policies. Raw data: **140 JSON files** (`FastCharge_*_structure.json`), one per test record.

Each JSON file includes: - **summary**: 16 per-cycle fields (capacity, energy, resistance, temperature, etc.) - **cycles_interpolated**: high-resolution voltage/current/temperature curves (not exported to CSV due to size)

3.2 Processed datasets

File	Rows	Description
data/cell_target s.csv	134	file_id, cell_id, EOL, initial_capacity
data/processed/c ycle_summary.csv	110,910	One row per cell per cycle; 18 columns

Notebooks: 01_data_exploration.ipynb (targets), 03_build_cycle_summary.ipynb (per-cycle export). Rebuild script: `scripts/rebuild_processed_data.py`.

End-of-life definition

- Initial capacity: first `cycle_index` ≥ 1 discharge capacity
- EOL: first subsequent cycle with `discharge_capacity` $< 0.8 \times \text{initial}$
- If threshold never reached: EOL = last cycle (censored)

Cleaning (Week 2)

Raw data has **140 files** but **134 unique barcodes** (five barcodes tested twice; one partial test). We apply **longest-run-wins dedupe**: keep the file with the higher EOL for each duplicate barcode; drop five short reruns and one formation-only partial cell (e1150800737381). Each retained row has a unique `file_id` (JSON filename) and `cell_id` (barcode). Full rule: `docs/week02/duplicate_barcode_policy.md`.

Data quality notes

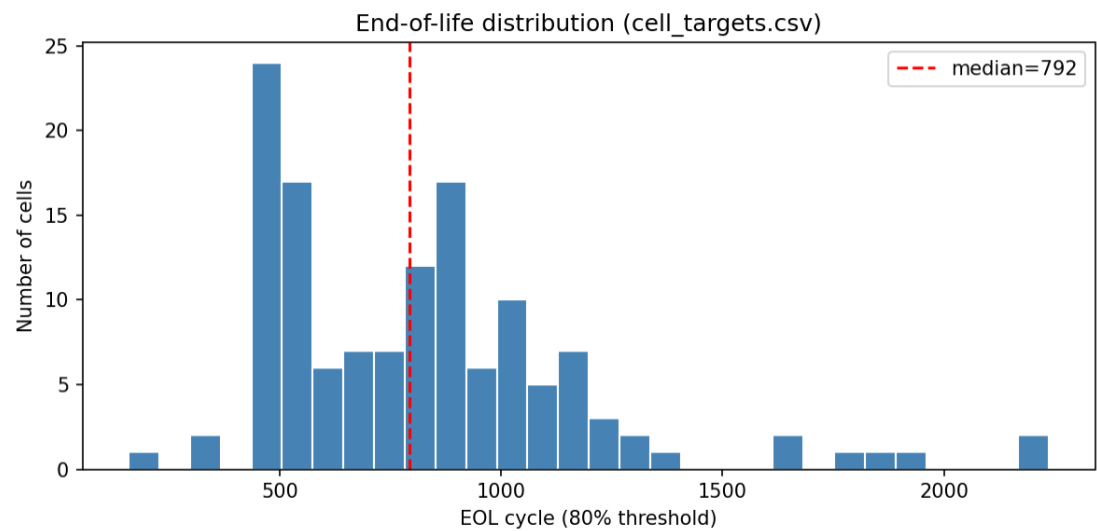
Issue	Action
Cycle 0	Formation/setup; exclude from baseline and early windows
Duplicate barcodes	Resolved — 134 cells after longest-run dedupe
Partial cell	e1150800737381 excluded (cycle 0 only)
Missing fields	charge_duration (1,209 rows); time_temperature_integrated (34 rows)

3.3 Exploratory analysis

End-of-life distribution

Across **134** labeled cells in `cell_targets.csv`:

Statistic	EOL (cycles)
Minimum	159
Median	792
Maximum	2,237
Mean	about 822

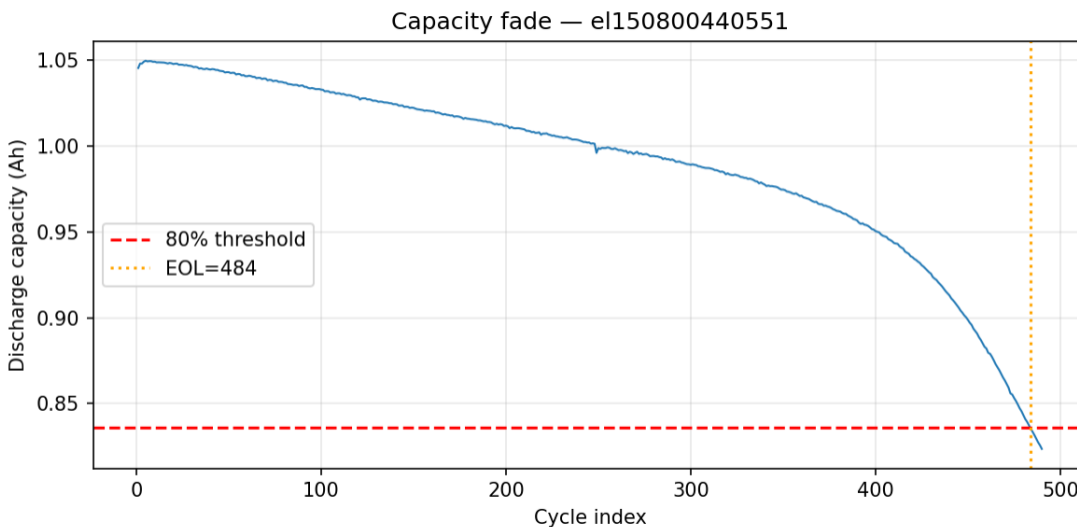


EOL distribution

The wide spread (159–2,237 cycles) matches the high variability reported by Severson et al. and motivates **cell-level** train/validation/test splits for modeling (no random row splits).

Example capacity fade

Figure below shows discharge capacity vs cycle for one cell (e1150800440551), with the 80% threshold and computed EOL marked. Capacity stabilizes near about 1.05 Ah after cycle 1; fade accelerates later in life.



Capacity fade example

Figures generated by `scripts/generate_week12_figures.py`.

Formation cycle (cycle 0)

Cycle 0 shows anomalously high discharge capacity and charge_duration compared to cycles 1+. All EOL and early-cycle analyses use `cycle_index ≥ 1`.

3.4 Planned preprocessing (before modeling)

1. **Done (Week 2):** Resolve duplicate `cell_id` entries.
2. **Done (Week 2):** Drop partial cells (e1150800737381 excluded).
3. **Done (Week 3):** Define early-cycle windows: cycles 1–20, 1–50, 1–100 for feature extraction (see §4).
4. **Outlier screening:** structural cleaning only (dedupe + partial-cell drop). We did **not** remove cells by statistical rules (e.g. extreme EOL or feature z-scores); the 159–2,237 cycle spread matches Severson et al. and is retained for modeling.

4. Feature Engineering

4.1 Overview

Week 3 builds a **cell-level feature matrix** for cycle-life prediction: one row per retained cell (134 after Week 2 dedupe), with hand-crafted measurements from the first 20, 50, and 100 cycles (`cycle_index` $\geq$ 1). Features follow Severson et al. (2019): per-cycle summary statistics plus $\Delta V(Q)$ voltage-curve differences from `cycles_interpolated`.

Outputs:

File	Rows	Description
data/processed/voltage_features.csv	134	10 $\Delta V(Q)$ features per cell
data/processed/cell_features.csv	134	Labels + 44 features (model input for Week 4)

Notebooks: 04_extract_voltage_features.ipynb, 05_build_cell_features.ipynb, 06_feature_correlation.ipynb. Index: docs/week03/README.md.

4.2 Summary features (`cycle_summary.csv`)

Extracted in 05_build_cell_features.ipynb from the 16-field per-cycle table (110,910 rows). **34 features** per cell:

Group	Count	Examples
Resistance	7	resistance_initial; mean and linear slope over w20 / w50 / w100
Capacity snapshots	6	capacity_c10, soh_c10, ... at cycles 10, 50, 100
Capacity windows	9	slope, mean, std of discharge capacity over w20 / w50 / w100
State of health	3	soh_w20, soh_w50, soh_w100 (capacity at window end $\div$ initial_capacity)
Energy efficiency	6	mean and std of energy_efficiency over w20 / w50 / w100 (dataset field; not Coulombic/Ah ratio)
Temperature	3	mean temperature_average over w20 / w50 / w100

Window notation: w20, w50, w100 denote statistics over cycles **1–20**, **1–50**, and **1–100** respectively. Linear slopes use ordinary least-squares fit vs cycle_index. SOH uses each cell's initial_capacity from cell_targets.csv.

4.3 Voltage $\Delta V(Q)$ features (raw JSON)

Following Severson et al., 04_extract_voltage_features.ipynb reads cycles_interpolated from each kept JSON file and computes discharge $\Delta V(Q) = V_{\text{late}}(Q) - V_{\text{early}}(Q)$ on the overlapping capacity range, interpolated to a common grid. Statistics per pair: mean, std, variance, min, max.

Cycle pair	Features (5 stats each)
10 → 50	delta_v_*_c10_c50
10 → 100	delta_v_*_c10_c100

10 voltage features total. Pairs align with the 50- and 100-cycle early windows used in summary features and planned ablations (Week 7).

4.4 Feature matrix

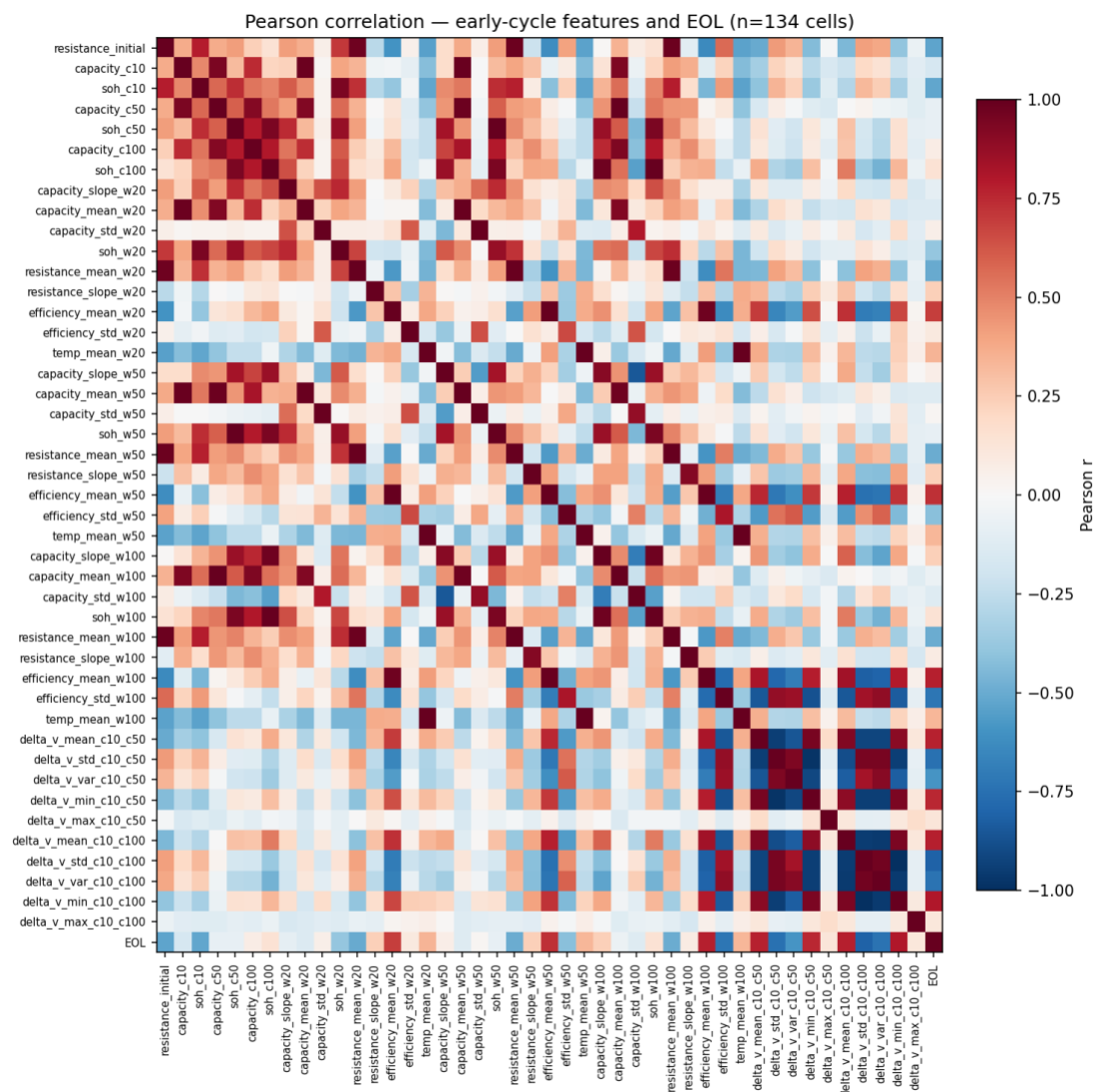
cell_features.csv merges:

- **Labels:** file_id, cell_id, EOL, initial_capacity
- **34** summary features (§4.2)
- **10** $\Delta V(Q)$ features (§4.3)

All 134 cells have complete feature rows (no missing values in the current cleaned dataset). Keys are unique on file_id (one test file per row).

4.5 Exploratory correlation with EOL

Before training models, 06_feature_correlation.ipynb computes **Pearson correlation** between each feature and EOL across all 134 cells, and saves a feature–feature heatmap including EOL.



Feature correlation heatmap

Strongest univariate associations ($|r| \approx 0.7\text{--}0.8$):

Feature	Pearson r	Interpretation
delta_v_std_c10_c100	-0.81	Greater spread in voltage-curve change (cycles 10→100) associates with shorter life
delta_v_min_c10_c100	+0.80	Higher minimum ΔV associates with longer life
efficiency_mean_w100	+0.78	Higher mean energy efficiency (cycles 1–100) associates with longer life

Weaker linear links: capacity snapshots and capacity slopes alone (e.g. `capacity_c100`, $|r| \approx 0.07$). This matches Severson et al.: **voltage-curve geometry and efficiency shift before obvious capacity fade** in the first about 100 cycles.

The heatmap also shows **high redundancy** among related features (blocks of correlated ΔV statistics and overlapping window summaries). This is expected and motivates **regularization or feature selection** in Week 4 (e.g. ElasticNet) rather than treating all 44 columns as independent inputs.

These correlations are **exploratory** (full dataset, univariate); predictive performance and feature importance will be assessed with held-out cells in §6.

4.6 Summary

We constructed 44 interpretable early-cycle features per cell—capacity, resistance, energy efficiency, temperature summaries over three windows, plus Severson-style $\Delta V(Q)$ curve differences. Univariate screening indicates $\Delta V(Q)$ and energy efficiency as the strongest linear signals for EOL on this dataset; raw capacity fade is a weaker solo predictor. The matrix `cell_features.csv` is the input for classical ML baselines in Week 4.

5. Methods

5.1 Machine learning baselines

We predict **EOL** (first cycle with `discharge_capacity < 0.8 × initial_capacity`, `cycle_index ≥ 1`) from the Week 3 feature matrix `cell_features.csv`: **134 cells, 44** early-cycle inputs, target EOL. Labels `file_id`, `cell_id`, and `initial_capacity` are excluded from X ; only EOL is the regression target.

Notebook: `notebooks/07_ml_baselines.ipynb`. Index: `docs/week04/README.md`.

Train / validation / test split

All splits are **cell-level** (by `file_id`) so no battery appears in more than one set:

Set	Cells	Role
Train	94 (about 70%)	Fit model weights
Validation	20 (about 15%)	Select hyperparameters
Test	20 (about 15%)	Final holdout metrics only

Split is fixed (`random_state=42`) and saved to `data/processed/cell_split.csv` for reuse in Week 5 (sequence model).

Models

Four feature-based regressors, increasing capacity:

1. **Linear regression** — ordinary least squares on standardized features (StandardScaler + LinearRegression).
2. **ElasticNet** — L1/L2-regularized linear model; grid search over $\alpha \in [1, 100]$ and $\text{l1_ratio} \in \{0.1, 0.5, 0.9, 0.95, 1.0\}$ on validation MAE. Weak penalties below $\alpha = 1$ were excluded because they failed to converge with 44 correlated features.
3. **Random forest** — RandomForestRegressor (300 trees); tune `max_depth` and `min_samples_leaf` on validation MAE. No feature scaling.
4. **XGBoost** — XGBRegressor (`objective='reg:squarederror'`, 300 trees, `subsample=0.8`, `colsample_bytree=0.8`); tune `max_depth` and `learning_rate` on validation MAE.

Linear / ElasticNet use a sklearn Pipeline with scaling; tree models use raw feature values.

Evaluation metrics

On each split we report:

- **MAE** — mean absolute error in cycles
- **RMSE** — root mean squared error (penalizes large misses)
- **MAPE** — mean absolute percentage error relative to true EOL

Primary comparison uses **test-set** metrics. Validation is used only for hyperparameter selection; test cells are never used during tuning.

Feature importance

For random forest and XGBoost we export **gain-based** (XGBoost) and **Gini impurity** (RF) importances for the top features (results/figures/feature_importance_*.png).

5.2 Sequence model

We predict **EOL** from **per-cycle trajectories** in `cycle_summary.csv` rather than the Week 3 feature matrix. Each cell contributes a sequence of length **N = 100** (cycles with `cycle_index` 1–100; cycle 0 excluded) and **four channels**: state-of-health (SOH = `discharge_capacity / initial_capacity`), `dc_internal_resistance`, `energy_efficiency`, and `temperature_average`. Tensor shape per split: (*n* cells, 100 timesteps, 4 channels). Target: EOL in cycles.

Notebook: notebooks/08_sequence_model.ipynb. Index: docs/week05/README.md.

Split: identical to 85.1 (data/processed/cell_split.csv, 94 / 20 / 20, random_state=42).

Preprocessing

1. **Input scaling** — StandardScaler on the four channels, fit on all timesteps from **train cells only**, applied to val and test.
2. **Target scaling** — EOL is standardized using train mean and standard deviation for the training loss; predictions are converted back to cycles before MAE / RMSE / MAPE.

Voltage-curve $\Delta V(Q)$ features (Week 3) are **not** included—they are derived from raw JSON, not `cycle_summary.csv`. The Week 4 XGBoost baseline used them; sequence-vs-tabular comparison therefore differs in input information as well as model class.

Model

GRU (Gated Recurrent Unit) regressor in PyTorch: unidirectional GRU (`batch_first=True`) reads timesteps in order; the final hidden state passes through dropout and a linear head to predict scaled EOL.

Setting	Value
Loss	Mean squared error on scaled EOL
Optimizer	Adam
Batch size	16
Early stopping	Validation MAE, patience 20, max 200 epochs

Hyperparameter selection

Grid search on **validation MAE** over 16 combinations:

- Hidden size $\in \{32, 64\}$
- Layers $\in \{1, 2\}$
- Dropout $\in \{0.1, 0.2\}$
- Learning rate $\in \{0.001, 0.0003\}$

Best validation settings: hidden size 64, 2 layers, dropout 0.2, learning rate 0.0003. Test-set metrics are reported once using this configuration; test cells are not used during tuning.

Evaluation metrics

Same as §5.1: **MAE**, **RMSE**, and **MAPE** on train, validation, and test splits (errors in cycles).

5.3 Monotonic SOH constraint

We extend the Week 5 GRU with a **dual-head** architecture that predicts both **EOL** and a **per-cycle SOH trajectory**, then compare **unconstrained** training to training with a **monotonic SOH penalty**—a light physics-informed regularizer, not a full physics-informed neural network (PINN).

Notebook: notebooks/09_monotonic_soh.ipynb. Index: docs/week06/README.md.

Split and preprocessing: identical to §5.2 (cell_split.csv, 94 / 20 / 20; input and EOL target scaling as in §5.2). True SOH trajectories (unscaled, capacity ÷ initial capacity) supervise the SOH head.

Dual-head model

The backbone is the same unidirectional GRU as §5.2 (hidden size 64, 2 layers, dropout 0.2, learning rate 0.0003—fixed at Week 5 best settings). Two linear heads read GRU hidden states:

1. **EOL head** — final timestep hidden state → scaled EOL (scalar per cell).
2. **SOH head** — hidden state at **each** timestep → predicted SOH at cycles 1–100.

Loss functions

Unconstrained ($\lambda = 0$): mean squared error (MSE) on scaled EOL plus $\alpha = 0.1$ times MSE on the SOH trajectory. The SOH term teaches the second head to match observed health curves; EOL remains the primary objective.

Constrained ($\lambda > 0$): add λ times a **monotonic penalty**—the batch mean of $\text{ReLU}(\text{predicted SOH at } t+1 \text{ minus predicted SOH at } t)$. The penalty is non-zero only when predicted SOH **increases** from one cycle to the next.

Setting	Value
Optimizer	Adam
Batch size	16
Early stopping	Validation EOL MAE , patience 20, max 200 epochs
SOH loss weight α	0.1
Monotonic weight λ	Grid {0.01, 0.1, 1.0}; select by validation EOL MAE

Hyperparameter selection

We grid-search λ on validation EOL MAE while holding GRU architecture and α fixed. Test-set metrics and SOH violation rates are reported once using the best λ ; test cells are not used during tuning.

Evaluation metrics

EOL: same as §5.1 — MAE, RMSE, MAPE in cycles on train, validation, and test.

SOH plausibility: count and rate of **monotonic violations**—consecutive cycle pairs where predicted SOH increases. Reported per split as violations ÷ (cells × 99 cycle pairs).

Example SOH trajectories (true vs unconstrained vs constrained) are plotted for test cells with the highest unconstrained violation counts.

5.4 Early-cycle window ablation

We ask how much early-cycle data is required for stable EOL prediction by retraining at window lengths $N \in \{20, 50, 100\}$ with the same cell-level split as §5.1–§5.3.

Notebook: notebooks/10_early_cycle_ablation.ipynb. Index: docs/week07/README.md.

Split: data/processed/cell_split.csv (94 / 20 / 20, random_state=42).

Tabular model (XGBoost)

For each N , we subset `cell_features.csv` to columns computable from cycles 1... N only (features_for_window):

N	Feature count	$\Delta V(Q)$ included
20	12	No
50	28	c10→c50 pair
100	44	c10→c50 and c10→c100 (Week 4 full set)

Hyperparameter grid and evaluation match §5.1 (validation MAE for max_depth and learning_rate; test metrics once per N).

Sequence model (GRU)

For each N , we build tensors (n cells, N timesteps, 4 channels) from `cycle_summary.csv`—same four channels as §5.2 (SOH, resistance, energy efficiency, temperature).

Hyperparameter grid (16 combinations) and early stopping match §5.2; best combo selected on validation MAE per window.

GRU training runs via `scripts/run_gru_ablation.py` in a fresh Python subprocess to avoid Jupyter kernel stale definitions and Mac PyTorch/OpenMP threading issues (documented in the notebook).

Week 6 dual-head monotonic GRU is **not** included; this ablation varies input window only.

Evaluation metrics

Same as §5.1: **MAE**, **RMSE**, **MAPE** on train, validation, and test. Primary comparison uses **test-set** MAE across N and model type. At $N = 100$, results should align with Week 4 XGBoost and Week 5 GRU baselines within rounding.

6. Results

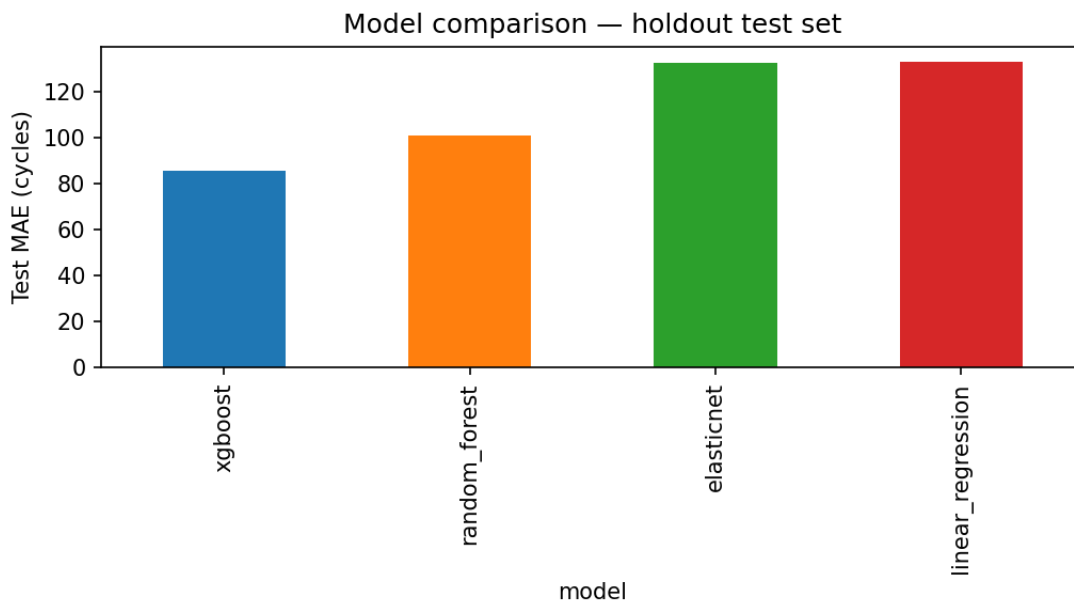
6.1 Machine learning baselines

We trained four regressors on `cell_features.csv` with the cell-level split in §5.1. Table 1 summarizes **holdout test** performance ($n = 20$ cells).

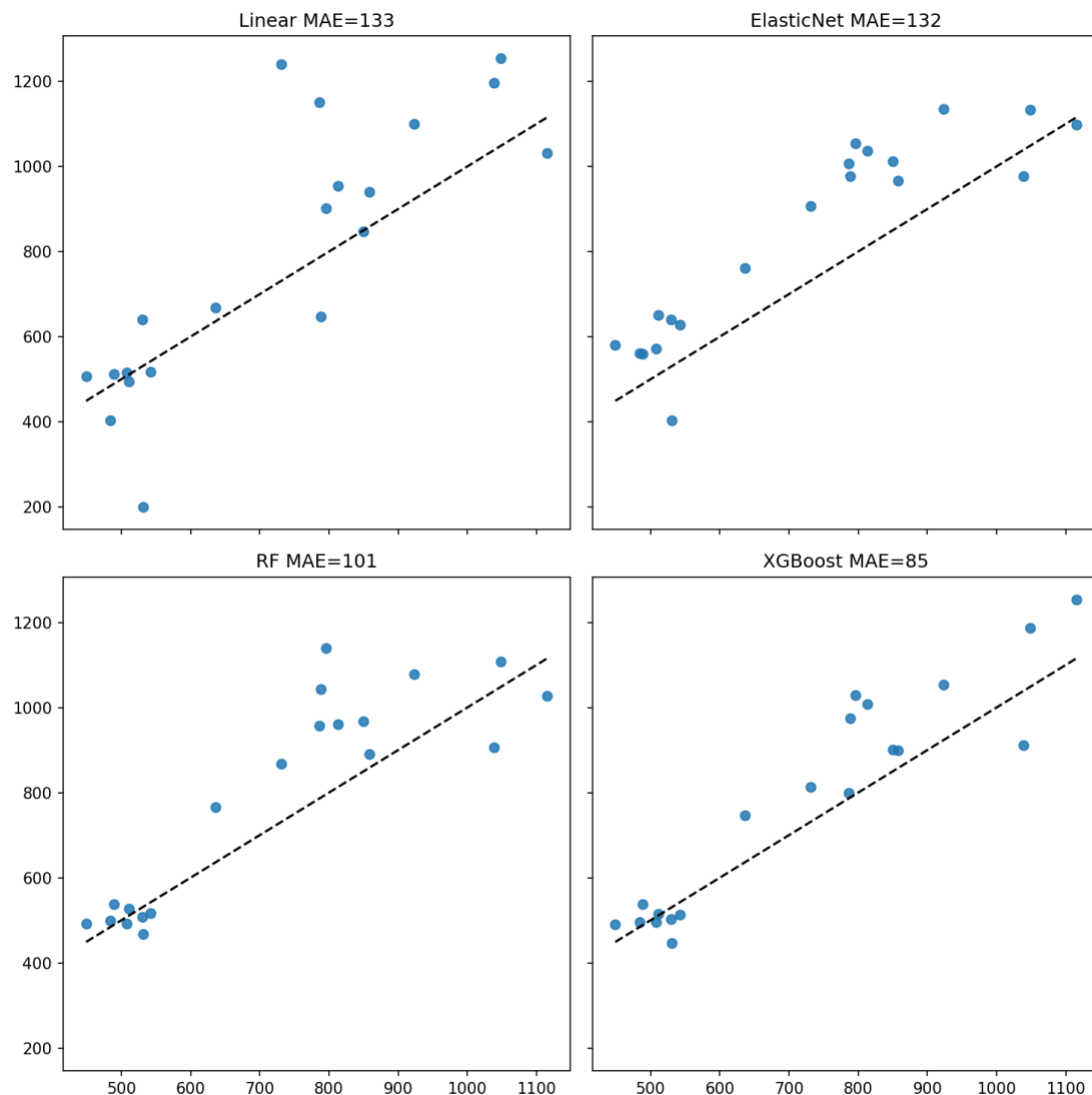
Table 1 — Test-set EOL prediction error

Model	MAE (cycles)	RMSE (cycles)	MAPE (%)
Linear regression	133	186	18.4
ElasticNet	132	147	19.3
Random forest	101	132	13.3
XGBoost	85	108	11.0

Best validation hyperparameters: ElasticNet $\alpha \approx 40$, $l1_ratio = 1.0$; random forest $max_depth = 5$, $min_samples_leaf = 1$; XGBoost $max_depth = 5$, $learning_rate = 0.1$.



Model comparison — test MAE



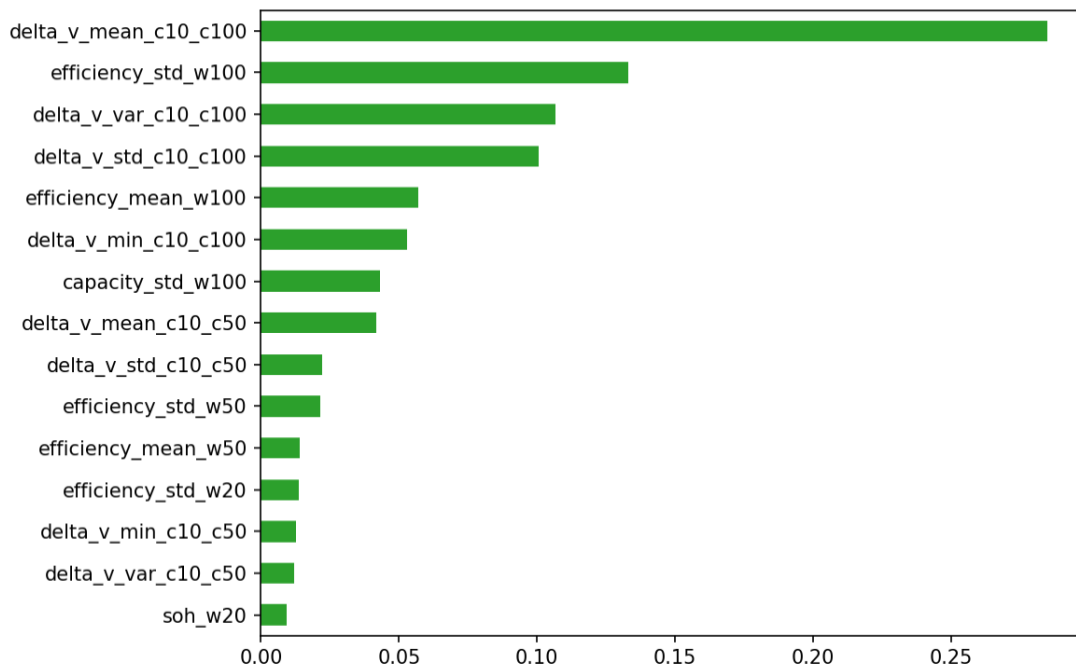
Predicted vs true EOL — all baselines (test set)

Interpretation

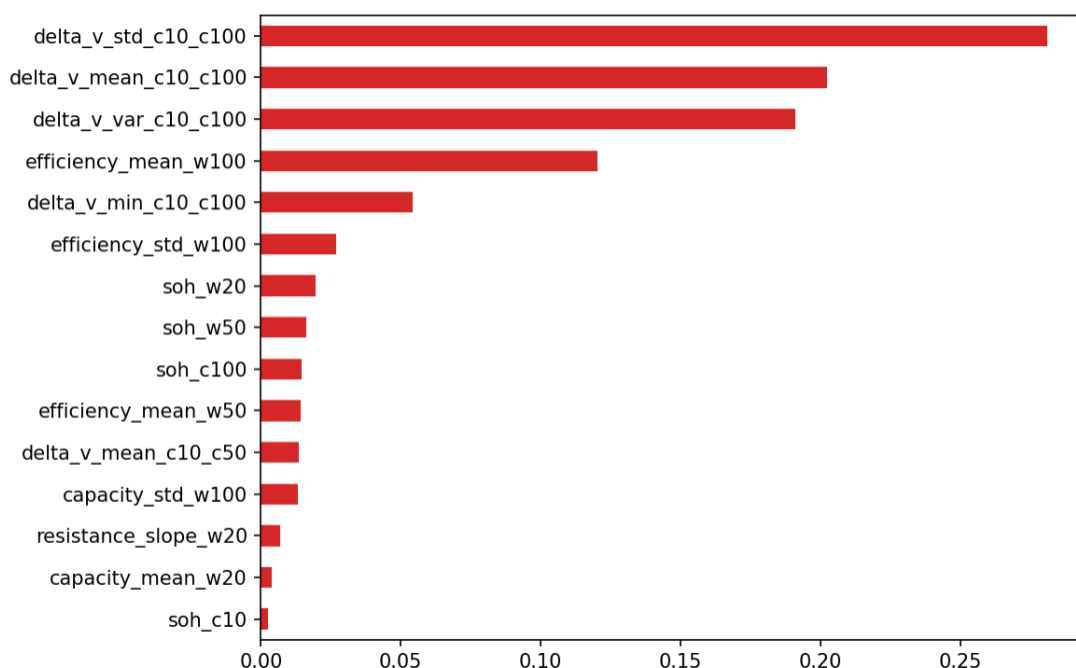
Tree ensembles outperform linear models on holdout. XGBoost achieves the lowest test MAE (about 85 cycles, about 11% MAPE), followed by random forest (about 101 cycles). Plain linear regression and ElasticNet both score about 132 cycles MAE on test despite ElasticNet improving validation error substantially (validation MAE about 70 vs about 280 for linear). That gap suggests high variance across small val/test folds rather than a stable ordering; with only 20 test cells, metrics should be interpreted cautiously.

Linear models overfit the training set. Training MAE for linear regression (about 85 cycles) is much lower than test (about 133), while ElasticNet's penalty raises training error (about 125) and improves validation behavior. This matches the Week 3 correlation heatmap: many redundant features motivate regularization or nonlinear models.

Feature importance aligns with exploratory correlation. Random forest and XGBoost rank $\Delta V(Q)$ statistics (e.g. `delta_v_mean_c10_c100`, `delta_v_var_c10_c100`) and **energy efficiency** summaries (`efficiency_std_w100`, `efficiency_mean_w100`) among the strongest predictors—consistent with §4.5 and Severson et al. (2019), where voltage-curve geometry and efficiency shift before obvious capacity fade.



Random forest feature importance (top 15)



XGBoost feature importance (top 15)

Limitations

- **Small holdout** ($n = 20$ test, $n = 20$ val) yields noisy metrics; a single mis-predicted short-life cell can inflate MAPE.
- **XGBoost fits training data nearly exactly** (train MAE ≈ 0), indicating memorization; test performance is still best among baselines but future work should monitor overfitting (e.g. stronger regularization, fewer trees).
- **No early-cycle window ablation yet** — all 44 features use up to 100 cycles; Week 7 varies $N = 20, 50, 100$ (*completed in §6.4*).

XGBoost is the **Week 4 reference baseline** for Week 5 sequence-model comparison.

6.2 Sequence model

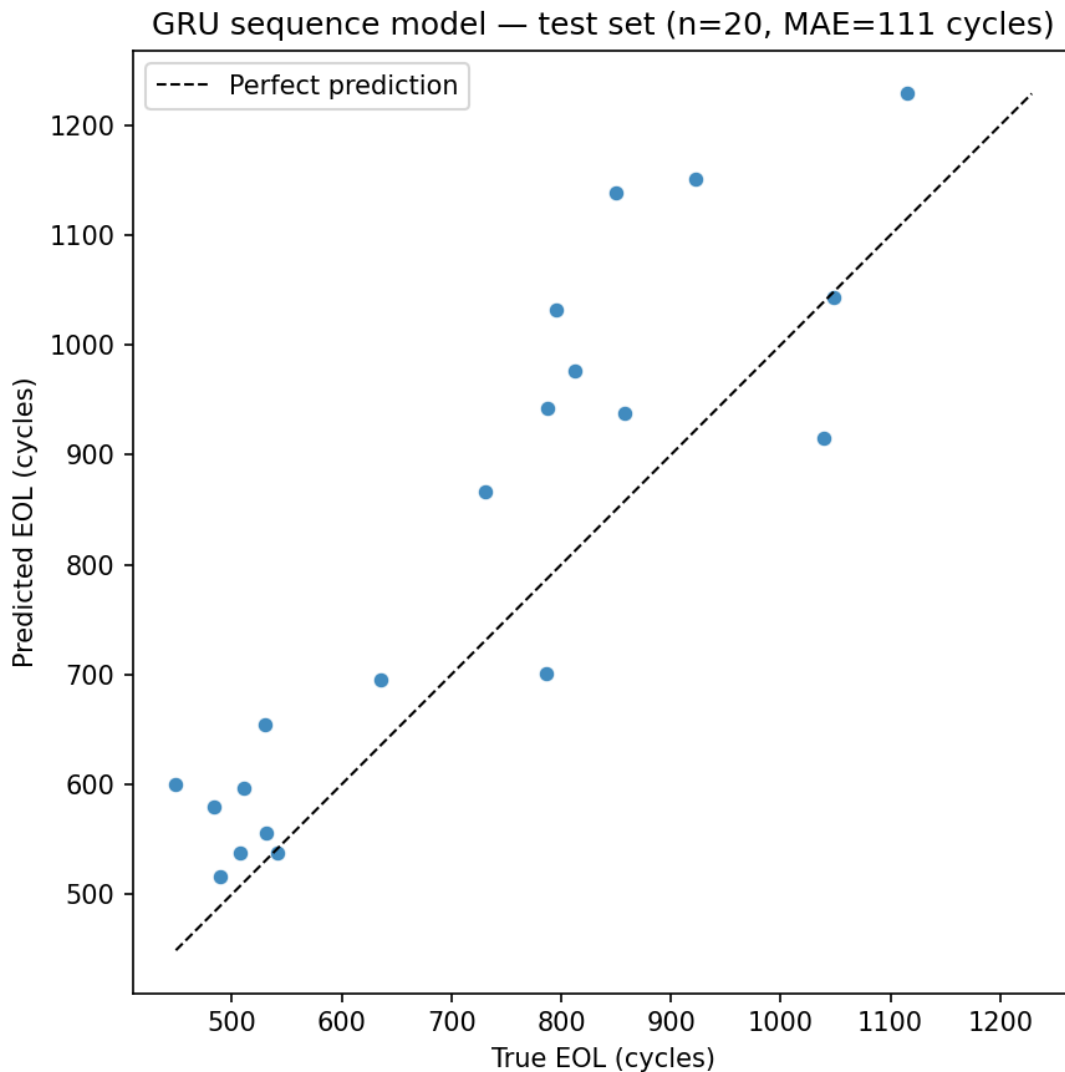
We trained the GRU sequence model in §5.2 on early per-cycle trajectories (100 cycles $\times$ 4 channels). Table 2 compares **holdout test** performance to the Week 4 XGBoost baseline ($n = 20$ test cells).

Table 2 — Test-set EOL prediction error (sequence vs best tabular baseline)

Model	MAE (cycles)	RMSE (cycles)	MAPE (%)
XGBoost (Week 4)	85	108	11.0
GRU sequence (Week 5)	111	134	15.4

Best validation hyperparameters (GRU): hidden size 64, 2 layers, dropout 0.2, learning rate 0.0003.

Full split metrics (GRU): train MAE 166, RMSE 255, MAPE 18.6%; val MAE 100, RMSE 129, MAPE 19.8%.



Predicted vs true EOL — GRU sequence model (test set)

Interpretation

XGBoost remains the stronger holdout model. The tuned GRU achieves test MAE about 111 cycles (about 15% MAPE) versus about 85 cycles (about 11% MAPE) for XGBoost—a gap of roughly 25 cycles on average. Validation MAE (about 100) is slightly better than test, suggesting modest fold variance rather than catastrophic overfitting, though train MAE (166) indicates the recurrent model does not fit the training set as tightly as XGBoost (train MAE ≈ 0).

Why the sequence model may lag. (1) Only **94** training cells for a neural sequence model with about 100 timesteps each. (2) Input channels exclude $\Delta V(Q)$ and other hand-crafted summaries that ranked highest in Week 4 feature importance. (3) Capacity and resistance change slowly in the first 100 cycles (SOH at cycle 100 still near 1.0 for many cells); much of the predictive signal in tabular models came from efficiency and voltage-curve statistics aggregated over the window.

Target scaling was required for training. Initial experiments without EOL standardization produced validation MAE >600 cycles (worse than predicting the train mean). Scaling the regression target during optimization improved validation MAE to about 100 cycles; this is noted as an implementation detail for small-data neural regression.

Limitations

- Same small holdout as §6.1 ($n = 20$ test).
- **Feature mismatch vs XGBoost** — not a controlled architecture-only ablation.
- Single sequence length ($N = 100$); Week 7 ablations vary the early window (see §6.4).
- No model checkpoint committed; metrics reproduced from notebooks/08_sequence_model.ipynb and results/metrics/gru_sequence.json.

The GRU is retained as the **Week 5 sequence baseline** for Week 6 (monotonic SOH constraint) and Week 7 comparisons.

6.3 Physics-informed constraint

We trained the dual-head GRU in §5.3 in **unconstrained** ($\lambda = 0$) and **constrained** (best λ from grid) modes. Table 3 summarizes **holdout test** EOL error ($n = 20$ cells); Table 4 reports SOH monotonic violations on the same split.

Table 3 — Test-set EOL prediction error (monotonic constraint vs baselines)

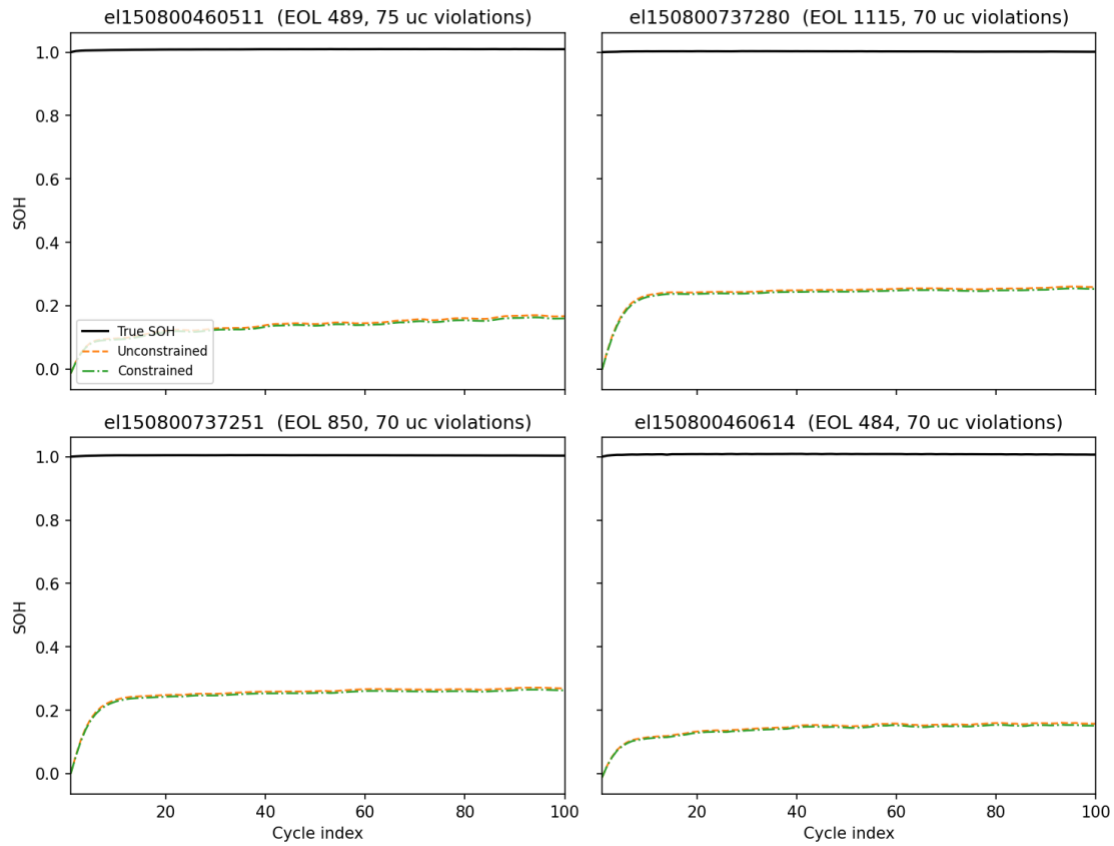
Model	MAE (cycles)	RMSE (cycles)	MAPE (%)
XGBoost (Week 4)	85	108	11.0
GRU dual-head, unconstrained	112	134	15.9
GRU dual-head, constrained ($\lambda = 1.0$)	112	134	15.9

Best λ (validation EOL MAE): 1.0 (grid {0.01, 0.1, 1.0}; validation MAE about 99.5 cycles for all three λ values within rounding).

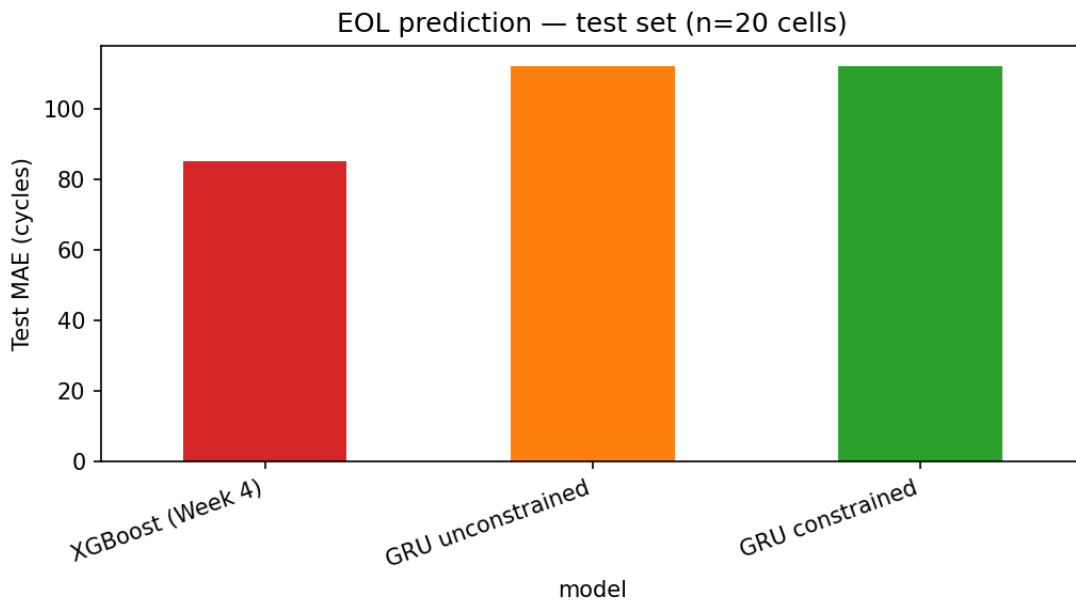
Table 4 — SOH monotonic violations (test set)

Model	Violations	Violation rate (%)
Unconstrained	1,199	60.6
Constrained ($\lambda = 1.0$)	1,191	60.2

Example test-set SOH trajectories (cycles 1-100)



Example SOH trajectories — test set



Model comparison — test EOL MAE

Interpretation

EOL error is essentially unchanged by the monotonic penalty. Unconstrained and constrained dual-head models both achieve test MAE about 112 cycles—within rounding of the Week 5 single-head GRU (about 111 cycles). Training and early stopping prioritize EOL; the SOH reconstruction term and monotonic penalty act on a separate output head whose gradients weakly affect the shared GRU representation.

The penalty slightly reduces SOH violations but not dramatically. Test violation rate falls from about 60.6% to about 60.2% of consecutive cycle pairs at $\lambda = 1.0$ —a drop of about 0.4 percentage points. Many violations correspond to small upward wiggles on an otherwise flat predicted curve in the first 100 cycles, where true SOH remains near 1.0; the tested λ values are too small to enforce visibly smoother trajectories without a larger trade-off against EOL or SOH reconstruction loss.

XGBoost remains the strongest holdout model. The monotonic constraint does not add input features ($\Delta V(Q)$ and other Week 3 summaries are still absent from the sequence). Closing the about 25-cycle gap to XGBoost would require richer inputs or a different modeling approach, not SOH regularization alone.

Limitations

- Same small holdout as §6.1–§6.2 ($n = 20$ test).
- λ grid limited to $\{0.01, 0.1, 1.0\}$; stronger penalties or post-hoc monotonic projection were not explored.
- EOL is predicted from the final hidden state, not derived from the SOH curve crossing 80%—the two outputs are only loosely coupled.
- Metrics reproduced from notebooks/`09_monotonic_soh.ipynb`, `gru_unconstrained.json`, and `gru_monotonic.json`.

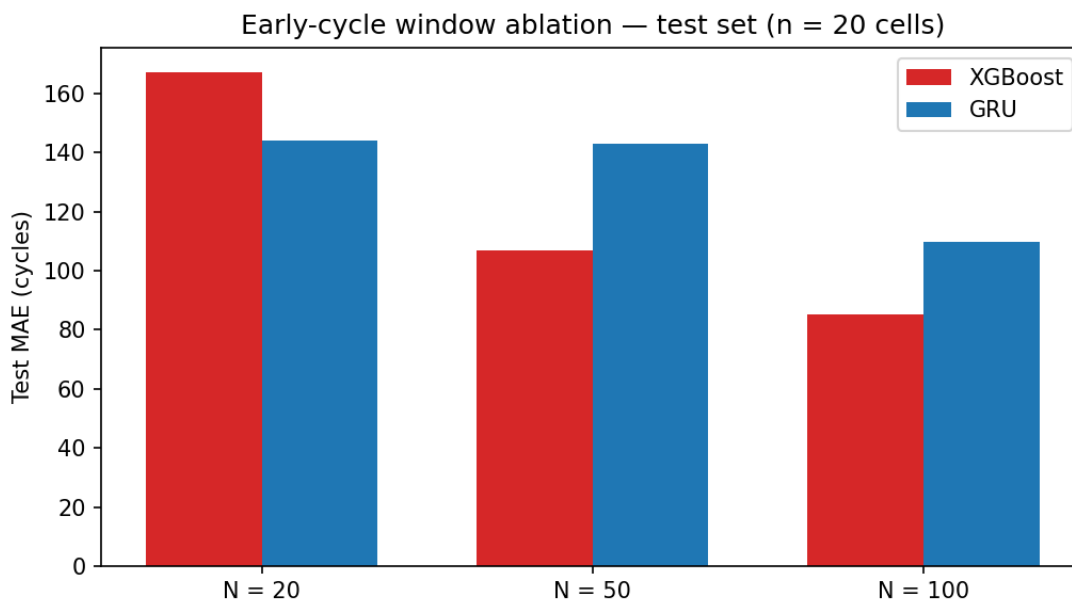
6.4 Ablation: early-cycle windows

We retrained XGBoost and the Week 5 single-head GRU at early-cycle windows $N \in \{20, 50, 100\}$ (§5.4). Table 5 summarizes **holdout test** EOL error ($n = 20$ cells).

Table 5 — Test-set EOL MAE by early-cycle window

N (cycles)	XGBoost MAE	XGBoost RMSE	XGBoost MAPE (%)	GRU MAE	GRU RMSE	GRU MAPE (%)
20	167	250	20.8	144	174	20.4
50	107	145	13.7	143	170	19.7
100	85	108	11.0	110	132	15.4

At $N = 100$, XGBoost test MAE (85 cycles) matches Table 1 (Week 4); GRU test MAE (110 cycles) matches Table 2 (Week 5, about 111 cycles)—confirming consistent splits and preprocessing.



Early-cycle window ablation — test MAE

Interpretation

XGBoost error drops sharply as N increases. Test MAE falls from about 167 cycles at $N = 20$ to about 107 at $N = 50$ and about 85 at $N = 100$. The step from $20 \rightarrow 50$ coincides with adding $\Delta V(Q)$ features (c10→c50); the step to 100 adds the c10→c100 voltage pair and full 100-cycle summaries. Tabular performance is therefore sensitive both to window length and to voltage-curve inputs absent from the sequence model.

GRU error is relatively flat between $N = 20$ and 50 (about 144 vs 143 cycles test MAE), then improves at $N = 100$ (about 110 cycles). With only 94 training cells, short sequences may under-utilize the recurrent architecture; the model approaches Week 5 performance only when the full 100-cycle input is available.

Model ranking reverses at the shortest window. At $N = 20$, GRU test MAE (about 144) beats XGBoost (about 167) because the tabular model loses all $\Delta V(Q)$ features and many aggregated statistics, while the GRU still reads per-cycle SOH, resistance, efficiency, and temperature. At $N = 100$, XGBoost again leads (about 85 vs about 110 cycles)—the same gap observed in §6.1–§6.2.

Limitations

- Same small holdout as §6.1–§6.3 ($n = 20$ test).
- Feature sets differ by N for XGBoost and by sequence length for GRU—not a single fixed feature vector with truncated rows.
- Week 6 monotonic dual-head GRU not evaluated at shorter windows.

- Metrics reproduced from notebooks/10_early_cycle_ablation.ipynb, scripts/run_gru_ablation.py, and results/metrics/xgboost_n*.json, gru_sequence_n*.json.

7. Discussion

7.1 Main findings

Across Weeks 4–7, **XGBoost on the full 44-feature matrix at $N = 100$ cycles** remains the strongest holdout model (test MAE about **85 cycles**, about **11% MAPE** on 20 test cells). A single-head **GRU** on 100-cycle $\times$ 4-channel trajectories achieves about **110 cycles** MAE—competitive but consistently worse on this dataset size.

The **Week 7 ablation** (§6.4) shows that **tabular performance depends strongly on window length and $\Delta V(Q)$ features**: XGBoost test MAE falls from about 167 cycles at $N = 20$ to about 85 at $N = 100$. The **GRU is less sensitive to moderate window changes** (about 144–143 cycles at $N = 20$ –50) and only approaches Week 5 levels at $N = 100$. At the shortest window, the sequence model outperforms XGBoost because voltage-curve summaries are unavailable to the tree model—a reminder that “best model” is **context-dependent** on how much early data and which feature types are available.

The **Week 6 monotonic SOH penalty** did not materially change EOL error (about 112 cycles test MAE constrained vs unconstrained) and only slightly reduced SOH violation rates. Physics-inspired regularization without richer inputs or a larger training set has limited impact here.

7.2 Limitations

- **Small dataset and holdout** — 134 cells total; 94 train / 20 val / 20 test. Metrics on 20 test batteries have high variance; a single outlier cell can move MAPE noticeably.
- **Single dataset and chemistry** — LFP/graphite, fast-charge protocol only (Severson et al., 2019); generalization to other chemistries or cycling conditions is untested.
- **Duplicate barcodes** — raw files contained duplicate cell IDs; we kept the longest continuous run per barcode (134 cells). See docs/week02/duplicate_barcode_policy.md. Residual protocol heterogeneity may remain.
- **Missing values** — charge_duration missing on about 1,209 cycle rows; time_temperature_integrated on 34 rows. These columns were not used in the Week 3 feature matrix or sequence channels.
- **Feature mismatch across model classes** — XGBoost uses hand-crafted summaries including $\Delta V(Q)$ from raw JSON; the GRU uses four channels from cycle_summary.csv only. Comparisons are informative but not a controlled architecture-only study.

- **Mechanism-agnostic features** — capacity fade and summary statistics do not identify degradation mode (Li et al., 2025).
- **EOL definition** — 80% of first measured capacity (`cycle_index ≥ 1`) is one convention; industry definitions vary.

7.3 Future work

- **External validation** — NASA or CALCE datasets (dos Reis et al., 2021) to test transfer beyond the MIT-Stanford-Toyota cohort.
- **Richer sequence inputs** — incorporate $\Delta V(Q)$ or other voltage-curve channels into recurrent models for a fairer comparison with XGBoost. Implementation options and effort estimates: `docs/future_work.md §1`.
- **Larger cohorts / ensembling** — more cells would stabilize neural sequence training and ablation trends.
- **Stronger physics-informed constraints** — larger λ grids, post-hoc monotonic projection, or full PINN-style formulations if SOH curve plausibility is a deployment requirement.
- **Alternative sequence architectures** — Transformers or pretrained time-series models (Chen et al., 2022; Hu et al., 2025) on longer or multi-resolution windows.

8. Conclusion

This eight-week GNEM project built a **reproducible Python pipeline** on the public MIT-Stanford-Toyota fast-charging dataset (Severson et al., 2019): 140 raw JSON files were cleaned to **134 unique cells**, per-cycle summaries and **44 early-cycle features** (including Severson-style $\Delta V(Q)$ voltage curves) were exported, and models were trained on a fixed **94 / 20 / 20** train/validation/test split (`random_state = 42`). Notebooks 01–10, committed processed tables, metrics JSON, and figures support rerunning the full analysis.

Answers to the research questions

1. **Can early-cycle data predict EOL before obvious fade?** Yes. The best holdout model (XGBoost at $N = 100$ cycles) achieves test MAE about **85 cycles** (about **11% MAPE**) predicting end-of-life on 20 unseen cells, using only measurements from the first 100 cycles.
2. **Which early-cycle features matter?** Exploratory correlation and tree importance consistently highlight **$\Delta V(Q)$** statistics and **energy efficiency** summaries. The Week 7 ablation shows that tabular error drops sharply when $\Delta V(Q)$ features become available ($N \geq 50$) and when the window reaches 100 cycles (test MAE about **167** → **107** → **85** cycles).
3. **Does a GRU outperform hand-crafted features? Not at the full window.** A single-head GRU on 100-cycle × 4-channel trajectories reaches about **110 cycles** test

MAE, behind XGBoost (about **85**). At $N = 20$ only, the GRU (about **144** cycles) beats XGBoost (about **167**) because voltage-curve summaries are unavailable to the tree model. The strongest model is **context-dependent** on window length and feature type.

4. **Does a monotonic SOH penalty help? Marginally for curve shape, not for EOL error.** A dual-head GRU with monotonic regularization leaves test EOL MAE near the unconstrained model (about **112** cycles) and only slightly lowers SOH violation rates. Physics-inspired regularization without richer inputs or more training data has limited impact here.

Summary

XGBoost on the full 44-feature matrix at $N = 100$ cycles is the best overall predictor in this study. Classical ensembles outperform linear baselines; the sequence model is competitive but does not close the gap on this cohort size. Early-cycle window length matters most for tabular models, while the GRU needs the full 100-cycle horizon to match its Week 5 performance.

Limitations

Results are bounded by a **small dataset** (134 cells, 20-cell test holdout), a **single chemistry and fast-charge protocol**, and **asymmetric inputs** (XGBoost uses $\Delta V(Q)$ from raw JSON; the GRU uses four summary channels only). Duplicate barcodes, missing fields in some cycle rows, and a fixed 80% EOL definition add further uncertainty. See §7.2 for detail.

Implications and future work

For **fast-charge cell screening**, observing about **50–100 early cycles** and extracting voltage-curve and efficiency features supports useful lifetime estimates before full degradation is visible, though metrics on 20 test cells should be treated as indicative rather than definitive. **External validation** on NASA or CALCE data, **richer sequence inputs** (e.g. $\Delta V(Q)$ channels), and **larger cohorts** are the most direct extensions; see §7.3 and docs/future_work.md.

9. References

1. K. A. Severson et al., “Data-driven prediction of battery cycle life before capacity degradation,” *Nature Energy*, vol. 4, pp. 383–391, 2019.
<https://doi.org/10.1038/s41560-019-0356-8>
2. Z. Fei et al., “Early prediction of battery lifetime via a machine learning based framework,” *Energy*, vol. 225, 120205, 2021.

<https://doi.org/10.1016/j.energy.2021.120205>

3. G. dos Reis et al., “Lithium-ion battery data and where to find it,” *Energy and AI*, vol. 5, 100081, 2021. <https://doi.org/10.1016/j.egyai.2021.100081>
4. D. Chen, W.-C. Hong, and X. Zhou, “Transformer Network for Remaining Useful Life Prediction of Lithium-Ion Batteries,” *IEEE Access*, vol. 10, pp. 19621–19628, 2022. <https://doi.org/10.1109/ACCESS.2022.3151975>
5. F. Wang et al., “Physics-informed neural network for lithium-ion battery degradation stable modeling and prognosis,” *Nature Communications*, vol. 15, 4332, 2024. <https://doi.org/10.1038/s41467-024-48779-z>
6. R. Li et al., “The importance of degradation mode analysis in parameterising lifetime prediction models of lithium-ion battery degradation,” *Nature Communications*, vol. 16, 2776, 2025. <https://doi.org/10.1038/s41467-025-57968-3>
7. J. Hu et al., “Early prediction of lithium-ion battery degradation with a generative pre-trained transformer,” *Nature Communications*, 2025. <https://doi.org/10.1038/s41467-025-66819-0>